

САНКТ-ПЕТЕРБУРГСКИЙ НАЦИОНАЛЬНЫЙ
ИССЛЕДОВАТЕЛЬСКИЙ УНИВЕРСИТЕТ ИТМО

Дисциплина: Фронт-энд разработка

Отчет
Лабораторная работа №3

Выполнил:
Шайкин Андрей
Группа
J3210

Проверил:
Добряков Д. И.

Санкт-Петербург
2026 г.

Задача

Целью данной лабораторной работы является миграция ранее разработанного веб-приложения на фреймворк **Vue.js 3** и реализация архитектуры *Single Page Application* (SPA).

Основные требования:

- Реализация маршрутизации средствами **Vue Router**.
- Использование компонентного подхода для построения интерфейса.
- Вынос бизнес-логики в переиспользуемые функции — **Composables**.
- Работа с внешним и локальным API через библиотеку **Axios**.
- Реализация системы защиты роутов (**Navigation Guards**).
- Рефакторинг CSS: избавление от *!important* и добавление плавных анимаций.
- Создание кастомных директив для интеграции сторонних библиотек.

Ход работы

На первом этапе проект был инициализирован с использованием сборщика *Vite*. Структура приложения была приведена к модульному виду: страницы вынесены в папку **views**, а переиспользуемые части интерфейса — в **components**. Для управления состоянием и побочными эффектами (запросы, темы) были созданы **Composables**.

Была настроена система маршрутизации. Для защиты страниц личного кабинета реализован навигационный хук **beforeEach**, который проверяет наличие токена доступа в *localStorage*. Если пользователь не авторизован, роутер автоматически перенаправляет его на страницу **/auth**.

Особое внимание было уделено реактивности. Логика фильтрации курсов в каталоге была реализована через **computed** свойства. Использование новой функции Vue 3.4+ **defineModel** позволило создать чистую двустороннюю связь между компонентом фильтров и списком курсов. Для улучшения визуального восприятия добавлена анимация **TransitionGroup**, обеспечивающая плавное перемещение карточек при фильтрации.

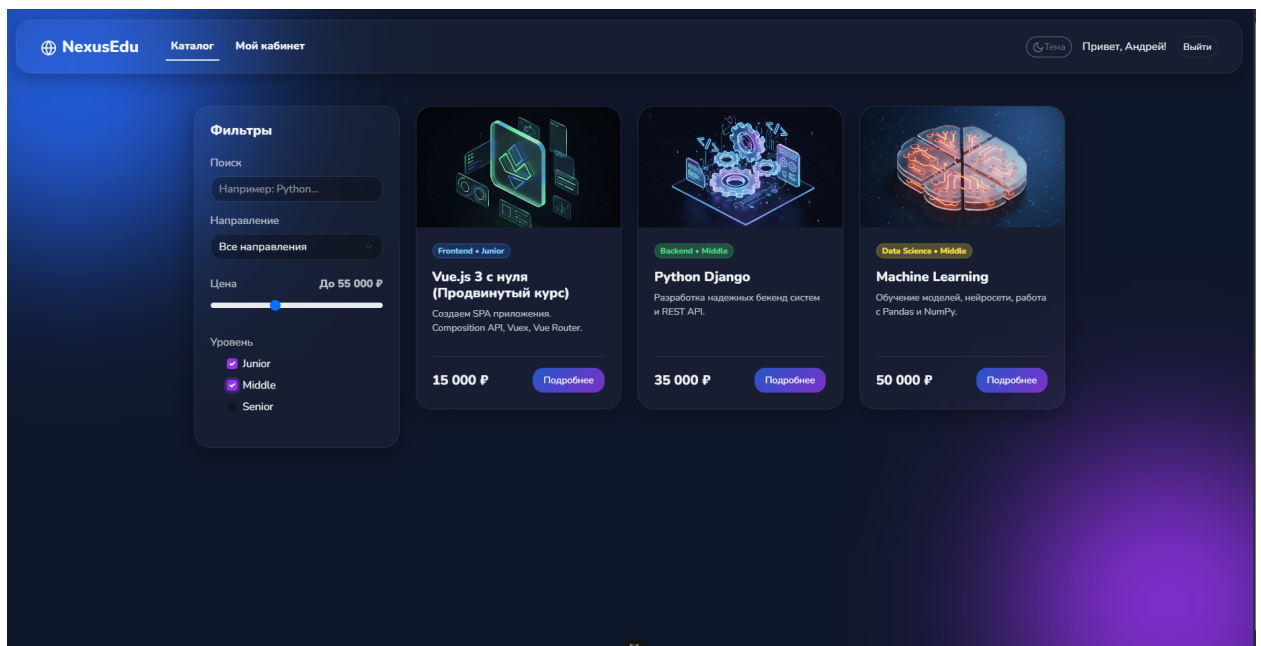


Рис. 1: Реактивный каталог с плавной анимацией фильтрации

Для работы с внешним *AdviceSlip API* и локальным сервером был создан настроенный инстанс **Axios** с интерцепторами, которые автоматически подставляют токен в заголовки. В профиле пользователя реализован механизм *Skeleton Loader*, предотвращающий резкие скачки интерфейса при загрузке данных из внешнего API.

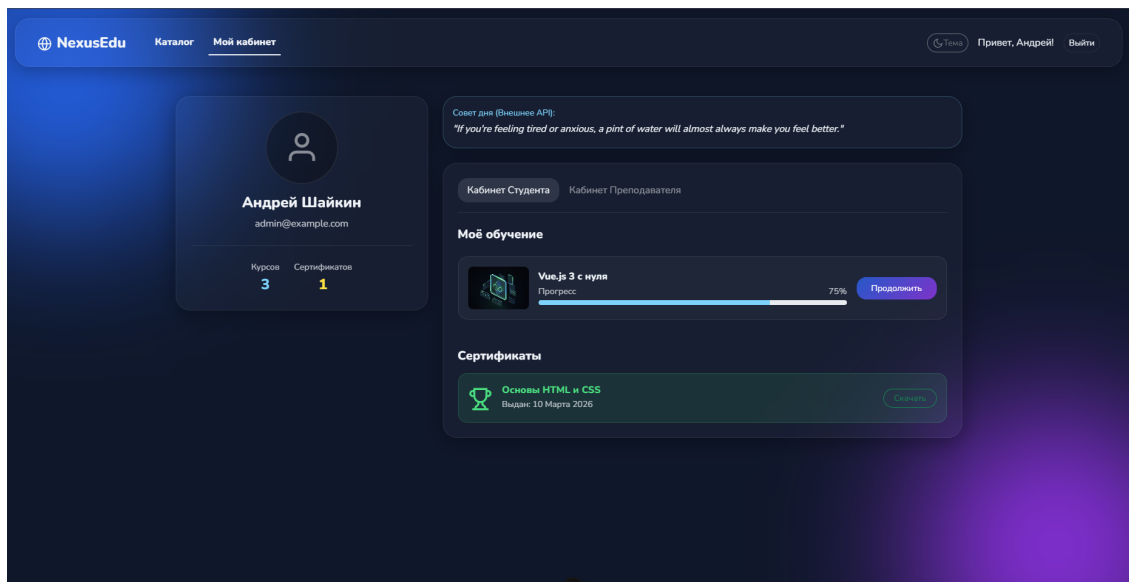


Рис. 2: Интеграция внешнего API и отображение личного кабинета

В завершение был проведен полный рефакторинг стилей. Все жесткие переопределения (*!important*) были заменены на настройку переменных Bootstrap 5. Для работы с тултипами была написана кастомная директива `v-tooltip`, которая корректно инициализирует и уничтожает объекты Bootstrap в жизненном цикле Vue-компонента.

Вывод

В ходе выполнения лабораторной работы №3 я освоил принципы построения современных SPA-приложений на базе фреймворка **Vue.js 3**.

Я научился применять декларативный подход к разработке интерфейсов, что позволило значительно упростить код по сравнению с предыдущими работами на ванильном JavaScript. Были освоены ключевые концепции Vue: реактивность (`ref`, `reactive`, `computed`), жизненный цикл компонентов, работа с **Vue Router** и компонентная декомпозиция.

Особую ценность составил опыт работы с **Composables**, что позволило разделить UI и бизнес-логику, сделав приложение масштабируемым. Также я закрепил навыки интеграции сторонних JS-библиотек через директивы и изучил способы оптимизации пользовательского опыта (UX) через плавные анимации и плейсхолдеры. В результате проект стал полноценным веб-приложением с чистой архитектурой и высокой производительностью.